

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«КРОС-ПЛАТФОРМНЕ ПРОГРАМУВАННЯ»

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірів: старший викладач кафедри АІС Бельська В. Ю.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 3

Тема: Створення структурованих типів даних на основі класів

Мета: отримати практичні навички у створенні класів, вкладених класів, використанні механізму наслідування при створенні нових типів даних.

Порядок виконання роботи:

Завдання 1. Описати клас для зберігання даних про тип об'єктів у заданій предметній області. При створенні класу дотримуватися принципу інкапсуляції – усі поля з даними повинні бути закритими, а для роботи з даними, що містяться у цих полях, реалізувати відповідні методи. Для створення екземплярів класу реалізувати відкриті конструктори, що заповнюють поля об'єктів даними. Реалізувати перевантаження одного – двох методів класу та конструктора класу. Для демонстрації функціонування класу створити масив з 5-7 екземплярів класу. Для генерування характеристик класу створити масиви можливих константних наборів даних та утилітний клас для випадкового вибору значення характеристик екземпляру та масиву екземплярів.

Для створеного масиву реалізувати:

- а) вивід на консоль даних з усіх елементів масиву;
- б) вивід даних лише тих елементів, які відповідають заданому у варіанті завдання критерію;
- с) пошук в масиві та вивід на консоль об'єктів з вказаною властивістю.

Варіанти завдань (обрати один довільний)

5) Предметна область: оренда автомобілів, клас: автомобіль, орієнтовний перелік полів: марка автомобіля, номерний знак, рік випуску, автомобіль орендовано (так/ні), дата оренди, термін оренди. Вивести окремо список вільних та окремо список орендованих автомобілів. Реалізувати пошук автомобілів, що звільняться з оренди у вказаному місяці.

Завдання 3. Створіть клас “Банк”, якому належить мережа АТМ банкоматів (має бути поле, що визначає кількість банкоматів). Визначити операції ініціалізації мережі банкоматів, тобто створення банкоматів та завантаження туди певної суми коштів. Розробіть метод визначення загальної суми грошей, що знаходиться у мережі банкоматів.

Розробіть клас “АТМ банкомат”. Сума, що є в наявності у банкоматі, має бути подана певними номіналами банкнот (1, 2, 5, 10, 20, 50 100, 200, 500 гривень). До кожного номіналу банкнот визначено кількість купюр даного номіналу. Крім цього, банкомат зберігає у своїх полях максимальну для видачі суму, а також максимальну кількість банкнот, яка може бути видана через віконце видачі готівки.

Реалізуйте метод ініціалізації банкомату (завантаження грошей до банкомату), метод ручного введення якоїсь суми через купюро приймач, метод зняття грошей у банкоматі. При визначенні номіналів банкнот під час видачі виходимо з мінімізації їхньої кількості. Для реалізації завдання побудуйте ієрархію класів-винятків користувача.

Завдання 1:

Car.java:

```
class Car {

    // приватні поля (інкапсуляція)
    private String brand;
    private String plate;
    private int year;
    private boolean isRented;
    private int rentMonth; // місяць оренди
    private int rentDuration; // термін оренди (в місяцях)

    // конструктор
    public Car(String brand, String plate, int year, boolean isRented, int
rentMonth, int rentDuration) {
        this.brand = brand;
        this.plate = plate;
        this.year = year;
        this.isRented = isRented;
        this.rentMonth = rentMonth;
        this.rentDuration = rentDuration;
    }

    // перевантажений конструктор (без оренди)
    public Car(String brand, String plate, int year) {
        this(brand, plate, year, false, 0, 0);
    }

    // гетери
    public boolean isRented() {
        return isRented;
    }
}
```

```

    public int getFreeMonth() {
        return rentMonth + rentDuration;
    }

    // перевантаження методу
    public void print() {
        System.out.println(brand + " | " + plate + " | " + year + " | " +
(isRented ? "Орендовано" : "Вільний"));
    }

    public void print(boolean detailed) {
        if (detailed) {
            System.out.println(brand + " | " + plate + " | " + year +
                " | оренда: " + isRented +
                " | місяць: " + rentMonth +
                " | термін: " + rentDuration);
        } else {
            print();
        }
    }
}

```

Utils.java:

```

import java.util.Random;

class Utils {

    static String[] brands = {"BMW", "Audi", "Toyota", "Ford", "Tesla"};

    static Random rand = new Random();

    public static String randomBrand() {
        return brands[rand.nextInt(brands.length)];
    }

    public static int randomYear() {
        return 2000 + rand.nextInt(25);
    }

    public static boolean randomRent() {
        return rand.nextBoolean();
    }

    public static int randomMonth() {
        return 1 + rand.nextInt(12);
    }

    public static int randomDuration() {
        return 1 + rand.nextInt(6);
    }
}

```

```
}  
}
```

Main.java:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Car[] cars = new Car[5];  
  
        // генерація масиву  
        for (int i = 0; i < cars.length; i++) {  
            cars[i] = new Car(  
                Utils.randomBrand(),  
                "AA" + i,  
                Utils.randomYear(),  
                Utils.randomRent(),  
                Utils.randomMonth(),  
                Utils.randomDuration()  
            );  
        }  
  
        // а) всі авто  
        System.out.println("Всі авто:");  
        for (Car c : cars) c.print();  
  
        // б) тільки вільні  
        System.out.println("\nВільні:");  
        for (Car c : cars)  
            if (!c.isRented())  
                c.print();  
  
        // б) орендовані  
        System.out.println("\nОрендовані:");  
        for (Car c : cars)  
            if (c.isRented())  
                c.print();  
  
        // в) пошук по місяцю  
        int searchMonth = 5;  
  
        System.out.println("\nЗвільняться у місяці " + searchMonth);  
        for (Car c : cars)  
            if (c.getFreeMonth() == searchMonth)  
                c.print(true);  
    }  
}
```

```
BCI АВТО:
Audi | AA0 | 2002 | Вільний
Tesla | AA1 | 2019 | Орендовано
BMW | AA2 | 2007 | Орендовано
Toyota | AA3 | 2003 | Вільний
Toyota | AA4 | 2018 | Вільний

ВІЛЬНІ:
Audi | AA0 | 2002 | Вільний
Toyota | AA3 | 2003 | Вільний
Toyota | AA4 | 2018 | Вільний

ОРЕНДОВАНІ:
Tesla | AA1 | 2019 | Орендовано
BMW | AA2 | 2007 | Орендовано

ЗВІЛЬНЯТЬСЯ У МІСЯЦІ 5
```

Рисунок 3.1 – Результат роботи скрипту завдання 1

Завдання 3:

Exception.java:

```
// базовий клас винятків
class ATMException extends Exception {
    public ATMException(String msg) {
        super(msg);
    }
}

// недостатньо коштів
class NotEnoughMoneyException extends ATMException {
    public NotEnoughMoneyException(String msg) {
        super(msg);
    }
}

// перевищення ліміту
class LimitExceededException extends ATMException {
    public LimitExceededException(String msg) {
        super(msg);
    }
}

// неможливо видати суму
class CannotDispenseException extends ATMException {
    public CannotDispenseException(String msg) {
        super(msg);
    }
}
```

ATM.java:

```
class ATM {

    private int[] nominal = { 500, 200, 100, 50, 20, 10, 5, 2, 1 };
    private int[] count = new int[nominal.length];

    private int maxWithdraw = 10000;
    private int maxNotes = 20;

    // ініціалізація
    public void loadMoney(int[] values) {
        for (int i = 0; i < count.length; i++) {
            count[i] = values[i];
        }
    }

    // КУПЮРОПРИЙМАЧ
    public void deposit(int[] values) {
        for (int i = 0; i < count.length; i++) {
            count[i] += values[i];
        }
    }

    // загальна сума
    public int getTotal() {
        int sum = 0;
        for (int i = 0; i < nominal.length; i++) {
            sum += nominal[i] * count[i];
        }
        return sum;
    }

    // зняття грошей
    public void withdraw(int amount) throws ATMException {

        if (amount > maxWithdraw)
            throw new LimitExceededException("Перевищено ліміт");

        if (amount > getTotal())
            throw new NotEnoughMoneyException("Недостатньо коштів");

        // копія масиву
        int[] tempCount = count.clone();

        int notesUsed = 0;

        for (int i = 0; i < nominal.length; i++) {

            while (amount >= nominal[i] && tempCount[i] > 0) {

                amount -= nominal[i];
```

```

        tempCount[i]--;
        notesUsed++;

        if (notesUsed > maxNotes)
            throw new LimitExceededException("Забагато купюр");
    }
}

if (amount > 0)
    throw new CannotDispenseException("Неможливо видати суму");

count = tempCount;

System.out.println("Гроші видано");
printStats();
}

// ручне введення суми через купюроприймач
public void depositAmount(int amount) throws ATMEException {

    if (amount <= 0)
        throw new ATMEException("Невірна сума");

    // тимчасовий масив
    int[] temp = new int[count.length];

    // розбиваємо суму на купюри (жадібний алгоритм)
    for (int i = 0; i < nominal.length; i++) {

        while (amount >= nominal[i]) {
            amount -= nominal[i];
            temp[i]++;
        }
    }

    // якщо залишок є — значить не можемо прийняти
    if (amount > 0)
        throw new CannotDispenseException("Неможливо прийняти таку суму");

    // додаємо в банкомат
    for (int i = 0; i < count.length; i++) {
        count[i] += temp[i];
    }

    System.out.println("Суму успішно внесено");
}

public void printState() {
    System.out.println("Стан банкомату:");
    for (int i = 0; i < nominal.length; i++) {
        System.out.println(nominal[i] + " грн: " + count[i] + " шт");
    }
}

```



```

        System.out.println("Загальна сума: " + getTotal());
    }
}

```

Bank.java:

```

class Bank {

    private ATM[] atms;

    public Bank(int count) {
        atms = new ATM[count];

        for (int i = 0; i < count; i++) {
            atms[i] = new ATM();
        }
    }

    public void init() {
        for (ATM atm : atms) {
            int[] money = {10,10,10,10,10,10,10,10,10};
            atm.loadMoney(money);
        }
    }

    public int getTotalMoney() {
        int sum = 0;
        for (ATM atm : atms) {
            sum += atm.getTotal();
        }
        return sum;
    }
}

```

Main.java:

```

public class Main {
    public static void main(String[] args) {

        ATM atm = new ATM();

        // 1. Ініціалізація
        atm.loadMoney(new int[]{2,2,2,2,2,2,2,2,2});
        atm.printState();

        // 2. Поповнення
        System.out.println("\nПоповнення...");
        atm.deposit(new int[]{1,0,0,0,0,0,0,0,0});
        atm.printState();

        // 3. Успішне зняття
    }
}

```

```
try {
    System.out.println("\nЗняття 880:");
    atm.withdraw(880);
    atm.printState();
} catch (ATMException e) {
    System.out.println(e.getMessage());
}

// 4. Перевищення ліміту
try {
    System.out.println("\nСпроба зняти 10001:");
    atm.withdraw(10001);
} catch (ATMException e) {
    System.out.println("Очікувана помилка: " + e.getMessage());
}

// 5. Недостатньо коштів
try {
    System.out.println("\nСпроба зняти 9999:");
    atm.withdraw(9999);
} catch (ATMException e) {
    System.out.println("Очікувана помилка: " + e.getMessage());
}

// 6. Неможливо видати суму
try {
    System.out.println("\nСпроба зняти 3:");
    atm.withdraw(3);
} catch (ATMException e) {
    System.out.println("Очікувана помилка: " + e.getMessage());
}
}
}
```

```
Стан банкомату:
200 грн: 2 шт
100 грн: 2 шт
50 грн: 2 шт
20 грн: 2 шт
10 грн: 2 шт
5 грн: 2 шт
2 грн: 2 шт
1 грн: 2 шт
Загальна сума: 1776

Поповнення...
Стан банкомату:
500 грн: 3 шт
200 грн: 2 шт
100 грн: 2 шт
50 грн: 2 шт
20 грн: 2 шт
10 грн: 2 шт
5 грн: 2 шт
2 грн: 2 шт
1 грн: 2 шт
Загальна сума: 2276

Зняття 880:
Гроші видано
Стан банкомату:
500 грн: 2 шт
200 грн: 1 шт
100 грн: 1 шт
50 грн: 1 шт
20 грн: 1 шт
10 грн: 1 шт
5 грн: 2 шт
2 грн: 2 шт
1 грн: 2 шт
Загальна сума: 1396
```

Рисунок 3.2 – Результат роботи скрипту завдання 3

```
Спроба зняти 10001:  
Очікувана помилка: Перевищено ліміт  
  
Спроба зняти 9999:  
Очікувана помилка: Недостатньо коштів  
  
Спроба зняти 3:  
Гроші видано  
Стан банкомату:  
500 грн: 2 шт  
200 грн: 1 шт  
100 грн: 1 шт  
50 грн: 1 шт  
20 грн: 1 шт  
10 грн: 1 шт  
5 грн: 2 шт  
2 грн: 1 шт  
1 грн: 1 шт  
Загальна сума: 1393
```

Рисунок 3.3 – Продовження результату роботи скрипту завдання 3

Висновок: на цій лабораторній роботі ми отримали практичні навички у створенні класів, вкладених класів, використанні механізму наслідування при створенні нових типів даних.